

Name _____

ERP _____

Instructions:

- Attempt all questions . Understanding question is part of quiz.
- write the final answer in the space provided for it **[1-18]**
- For each of the following program segments, assume that the code is contained within the main function and appropriate headers are also declared

1. What is the output of the following code? [1 point]

```
int arr[] = {1, 2, 3, 4, 5};  
int* ptr = arr + 2;  
ptr --;  
cout << *ptr;
```

- a) 1
- b) 2
- c) 3
- d) 4

2. Given the following code, what happens? [1 point]

```
char* str = "Hello";  
str[0] = 'h';
```

- a) Changes the first character to 'h'
- b) Compiles but causes runtime error
- c) Compilation error
- d) Changes nothing but runs successfully

3. What is the value of x after this code executes? [1 point]

```
void updateValue(int* p) {  
    static int val = 5;  
    *p = val++;  
}
```

```
int main() {  
    int x = 10;  
    updateValue(&x);  
    updateValue(&x);  
    return 0;  
}
```

- a) 5
- b) 6
- c) 10
- d) 11

4. In the following code, what is sizeof(arr) inside the function? [1 point]

```
void processArray(int arr[]) {  
    cout << sizeof(arr);  
}  
  
int main() {  
    int numbers[5] = {1,2,3,4,5};  
    processArray(numbers);  
}
```

- a) 20 (assuming int is 4 bytes)
- b) 5
- c) 4 or 8 (size of pointer)
- d) Compilation error

5. What happens when this code is executed? [1 point]

```
char* func() {  
    char str[] = "Test";  
    return str;  
}
```

- a) Returns "Test" successfully
- b) Compilation error
- c) Returns a dangling pointer
- d) Returns null pointer

6. What will be printed? [1 point]

```
static int arr[] = {1, 2, 3};  
void printFirst() {    cout << arr[0];    arr[0]++;}  
int main() {  
    printFirst();  
    printFirst();  
}
```

- a) 1 1
- b) 1 2
- c) 2 2
- d) Undefined behavior

7. Which statement about string literals is correct? [1 point]

- a) They can be modified using array syntax
- b) They are stored in stack memory
- c) They are stored in read-only memory
- d) Their size can be changed at runtime

8. What's wrong with this code? [1 point]

```
int* getArray() {  
    int numbers[5] = {1,2,3,4,5};  
    return numbers;}  
a) Array size is too small  
b) Returns address of local variable  
c) Array initialization is incorrect  
d) Nothing is wrong
```

9. What happens here? [1 point]

```
void swap(int* x, int* y) {  
    int* temp = x;  
    x = y;  
    y = temp;}  
  
int main() {  
    int a = 5, b = 10;  
    swap(&a, &b);  
    cout << a << b;}
```

- a) Values are swapped
- b) Pointers are swapped but values remain unchanged
- c) Compilation error
- d) Runtime error

10. What is the output? [1 point]

```
void modifyString(char str[]) {    str = "New";}  
int main() {  
    char text[] = "Old";    modifyString(text);    cout << text;}
```

- a) New
- b) Old
- c) Compilation error
- d) Runtime error

11. Consider this code: [1 point]

```
int arr[3][4];  
int (*ptr)[4] = arr;
```

What is ptr?

- a) Pointer to an integer
- b) Array of 4 pointers
- c) Pointer to an array of 4 integers
- d) Array of 3 pointers

12. What's the output? [1 point]

```
static int count = 0;
void increment(int& x) {
    static int factor = 2;
    x = x + (count++ * factor);}
int main() {
    int num = 5;    increment(num);    increment(num);    cout << num;}
```

- a) 5
- b) 7
- c) 9
- d) 11

13. What happens here? [1 point]

```
int main() {
    char str1[] = "Hello";
    char* str2 = "Hello";
    str1[0] = 'h';
    str2[0] = 'h';}
```

- a) Both modifications succeed
- b) Both modifications fail
- c) First succeeds, second causes runtime error
- d) Compilation error

14. What's the output? [1 point]

```
void process(int arr[], int size) {
    static int last_pos = 0;
    arr[last_pos++ % size] *= 2;
}
int main() {
    int nums[] = {1,2,3};
    process(nums);
    process(nums);    cout << nums[0] << nums[1]; }
```

- a) 1 2
- b) 2 4
- c) 2 2
- d) 1 4

15. Complete the following function declaration to receive a 2D array of integers with 3 columns and variable number of rows. The function should return a pointer to an array containing the sum of each row.[4]

```
_____ calculateRowSums(_____) {  
    // Function implementation not needed  
}  
  
int main() {  
    int matrix[][3] = {{1,2,3}, {4,5,6}, {7,8,9}};  
    int* sums = calculateRowSums(matrix, 3); // 3 is number of rows  
}
```

16. What will be the output of the following code segment? If there's an error, explain why.

[3 Points]

```
int arr[] = {10, 20, 30, 40, 50};  
  
int* ptr = arr + 2;  
  
ptr -= 1;  
  
cout << *ptr << " " << *(ptr + 2);
```

17. What is the output of the following program? Explain the role of the static variable[4 Points]

```
void countCalls() {  
    static int count = 0;    count++;    cout << "Function called " << count << " times\n"; }  
  
int main() {  
    for(int i = 0; i < 3; i++) {  
        countCalls();    } } }
```

18. For each of the following code segments, specify the output or identify any errors and propose solution: [15]

a)

```
int arr[] = {1, 2, 3, 4, 5};  
int* p1 = arr;  
int* p2 = &arr[3];  
cout << p2 - p1;
```

b)

```
void modifyArray(int* arr, int size) {  
    static int lastIndex = 0;  
    arr[lastIndex++] = 100;  
}  
  
int main() {  
    int numbers[] = {1, 2, 3, 4};  
    modifyArray(numbers, 4);  
    modifyArray(numbers, 4);  
    cout << numbers[0] << " " << numbers[1];  
}
```

c)

```
int* createArray() {  
    int arr[] = {1, 2, 3};    return arr; }  
  
int main() {  
    int* ptr = createArray();  
    cout << ptr[0];}
```

d)

```
int* doubleValues(int arr[], int size) {  
    int result[10];  
    for(int i = 0; i < size; i++) {  
        result[i] = arr[i] * 2;    }    return result; }  
  
int main() {  
    int numbers[] = {1, 2, 3, 4, 5};  
    int* doubled = doubleValues(numbers, 5);  
    cout << doubled[0] << " " << doubled[1];}
```

e)

```
int* trackMax(int arr[], int size) {  
    static int maxSeen = 0;  
    for(int i = 0; i < size; i++) {  
        if(arr[i] > maxSeen)  
            maxSeen = arr[i];    }  
    return &maxSeen; }  
  
int main() {  
    int test1[] = {1, 5, 3};  
    int test2[] = {2, 4, 7};  
    int* max1 = trackMax(test1, 3);  
    cout << *max1 << endl;  
    int* max2 = trackMax(test2, 3);  
    cout << *max1 << " " << *max2; }
```

19. Coding Part [10 Marks Each]

- 1. Write a program that finds the sum of alternate elements in an array (elements at index 0, 2, 4, etc.) using pointer arithmetic.**

```
int* getAlternateSum(int arr[], int size) {  
    // Should return static array with:  
  
    // result[0] = sum of elements at even indices (0,2,4...)  
    // result[1] = sum of elements at odd indices (1,3,5...)  
  
    // Use pointer arithmetic instead of array indexing  
}  
  
int main() {  
    int numbers[] = {1, 2, 3, 4, 5, 6};  
    int* sums = getAlternateSum(numbers, 6);  
  
    //Print : Sum at even indices: 9 (1+3+5)  
    // Print: Sum at odd indices: 12 (2+4+6)  
}
```

- 2. Create a function that counts vowels and consonants in a string using character array manipulation.**

```
void countLetters(const char* str, int* vowels, int* consonants) {  
    // Count vowels and consonants in str  
  
    // Store results in vowels and consonants  
  
    // Handle both uppercase and lowercase letters }  
  
int main() {  
    char text[] = "Hello World";  
  
    int vowelCount = 0, consonantCount = 0;  
  
    countLetters(text, &vowelCount, &consonantCount);  
  
    // Should print number of vowels and consonants  
}
```


- 3. Write a function that calculates the sum of each row in a 2D array and stores them in a static array.**

```
int* getRowSums(int matrix[][3], int rows) {  
    // Calculate sum of each row  
    // Return static array containing sums  
    // Use pointer arithmetic where possible }  
  
int main() {  
    int matrix[][3] = {  
        {1, 2, 3},  
        {4, 5, 6},  
        {7, 8, 9}    };  
    int* sums = getRowSums(matrix, 3);  
    // Should print: 6, 15, 24 }
```

- 4. Create two functions that share a static variable to track number of positive and negative numbers processed.**

```
void processNumber(int num) {  
    // Use static variables to count positive and negative numbers  
    // Print current counts after processing each number  
}  
  
int* getStats() {  
    // Return static array containing:  
    // [0] = count of positive numbers  
    // [1] = count of negative numbers }  
  
int main() {  
    processNumber(5); // Positive: 1, Negative: 0  
    processNumber(-2); // Positive: 1, Negative: 1  
    processNumber(7); // Positive: 2, Negative: 1  
    processNumber(-4); // Positive: 2, Negative: 2    int* stats = getStats();    // Print final statistics }
```

20. Logical Questions [20 marks]

1. [2 marks]

```
int* processArray() {  
    int localArr[] = {1, 2, 3};  
    static int staticArr[] = {1, 2, 3};  
    return staticArr; // Line A  
    return localArr; // Line B }
```

Explain why returning localArr (Line B) would cause problems while returning staticArr (Line A) works correctly. Draw the memory layout to support your answer and explain what happens to both arrays when the function returns.

2. [6 marks]

Given the following code:

```
inline int squareInline(int x) {  
    return x * x;}  
int squareNormal(int x) {  
    return x * x;}  
  
int main() {  
    int a = 4, b = 5;  
    int result1 = squareInline(a);  
    int result2 = squareNormal(b);  
  
    cout << "Square of " << a << " using inline function: " << result1 << endl;  
    cout << "Square of " << b << " using normal function: " << result2 << endl;  
    return 0;}
```

- What will be the output of the program?
- What is the difference in behavior between the inline function squareInline() and the normal function squareNormal()?
- Explain the possible performance difference between calling squareInline() and squareNormal().

3. [8 marks]

```
char* str1 = "Hello";  
char str2[] = "Hello";  
str1[0] = 'h'; // Line 1  
str2[0] = 'h'; // Line 2
```

1. Where are str1 and str2 stored in memory?
2. Why does Line 1 cause a runtime error while Line 2 works fine?
3. Explain why we can't modify string literals.
4. What happens if we don't modify but just read str1[0] and str2[0]?

5. [3 Marks]

```
void countCalls() {  
    static int count = 0;  
    int regularCount = 0;  
    count++;  
    regularCount++;  
    cout << "Static: " << count << ", Regular: " << regularCount << endl;  
}
```

If this function is called three times:

1. Explain the initialization and lifetime of both variables
2. What will be printed in each call and why?
3. What would happen if static initialization was: static int count = rand() % 10;

5. [1 Mark]

Consider these function declarations:

```
void func1(int arr[10]);  
void func2(int* arr);
```

```
void func3(int (&arr)[10]);  
  
int main() {  
    int numbers[10] = {1,2,3,4,5,6,7,8,9,10};  
    cout << sizeof(numbers) << endl; // Line 1  
    func1(numbers);  func2(numbers);  func3(numbers); }  
}
```

Explain:

1. Why is sizeof(numbers) in main different from sizeof(arr) inside func1 and func2?